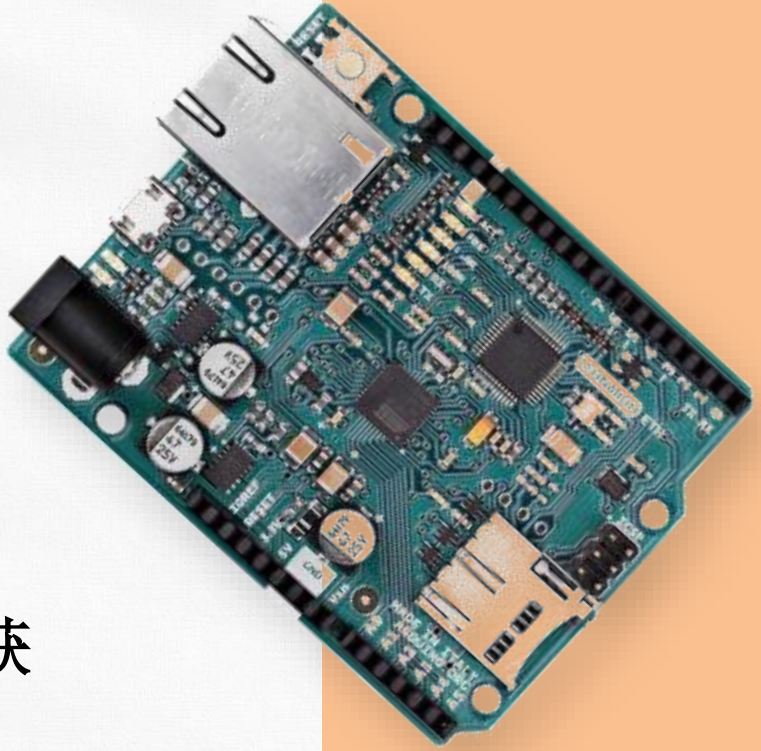
单片机



第6章：PWM输出和输入捕获

目 录

CONTENTS

01. PWM简介

02. PWM相关的HAL库接口函数及应用

03. 输入捕获简介

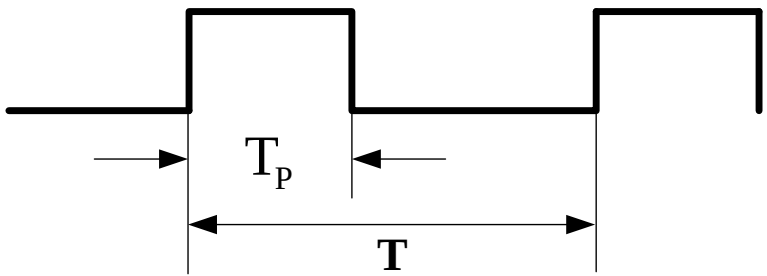
04. 输入捕获的HAL库接口函数及应用

01

PWM简介

PWM简介

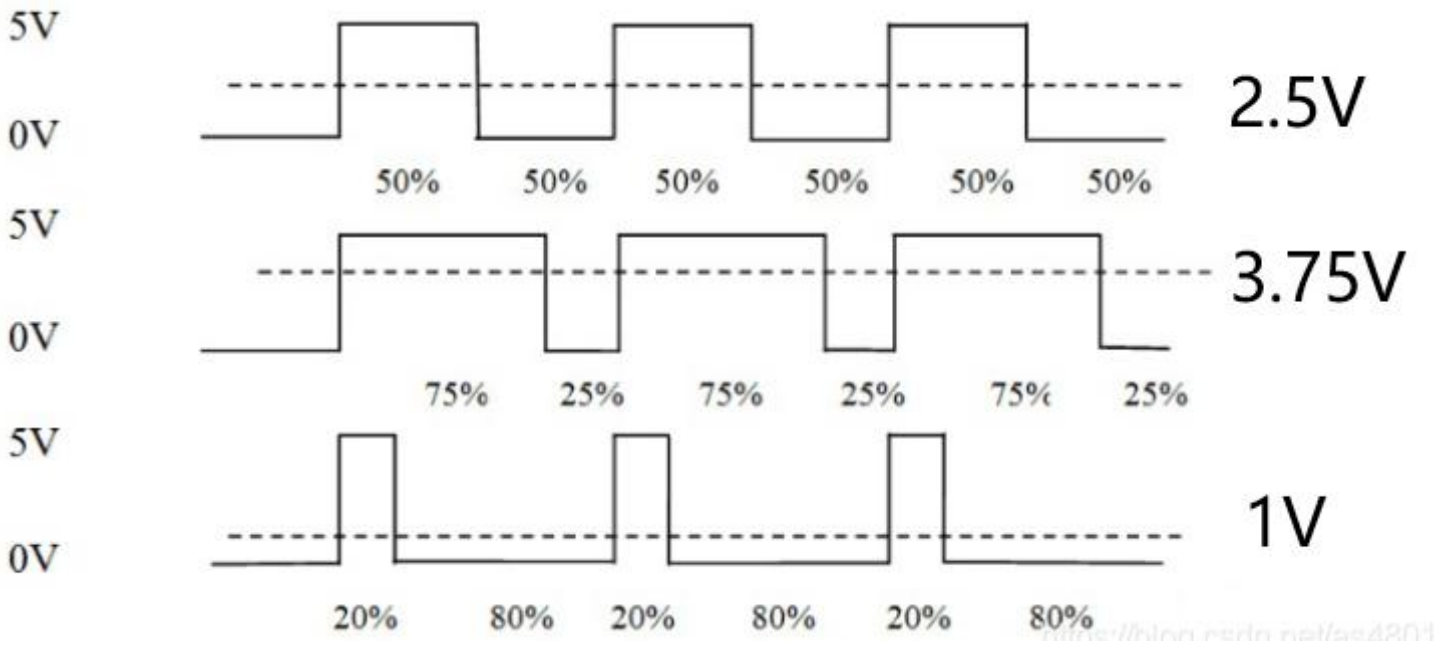
脉冲宽度调制（PWM，Pulse Width Modulation），是一种利用微处理器的数字输出来对模拟电路进行控制的技术。通过**控制占空比，输出一系列幅值相等、单位周期内脉冲宽度不相等的脉冲**，用这些脉冲长短来代替所需要的波形高度。主要用于电机调速、节能等特性。



占空比是指在一个周期内，高电平时间占整个信号周期的百分比，即高电平时间与周期的比值：占空比=Tp/T

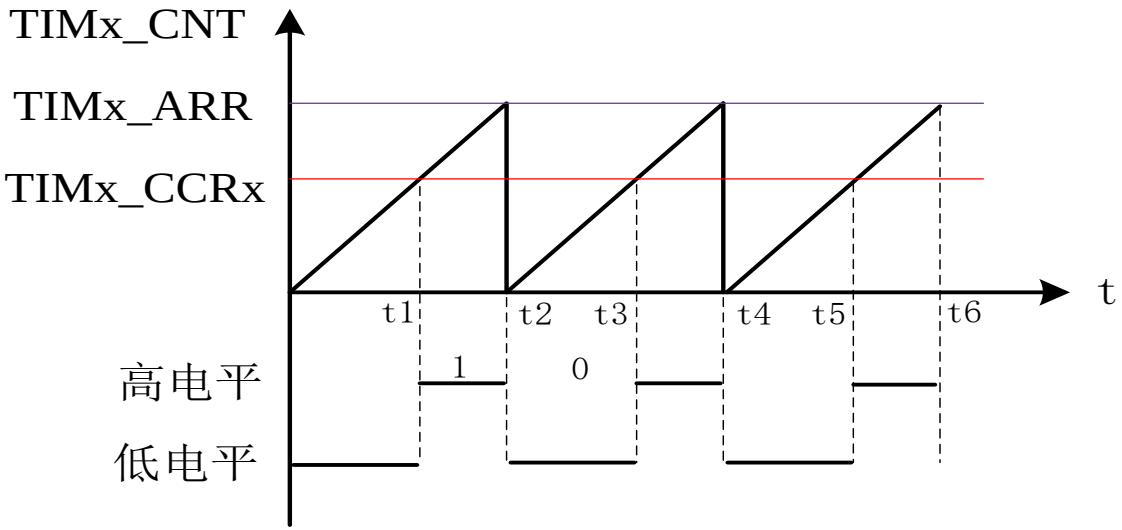
其中周期或频率：T或f=1/T

PWM简介



不同占空比的PWM波形

PWM实现原理



- 在PWM的一个周期内，定时器从0开始向上计数，在0-t1时间段，定时器计数器TIMx_CNT值小于TIMx_CCRx值，输出低电平；
- 在t1-t2时间段，定时器计数器TIMx_CNT值大于TIMx_CCRx值，输出高电平；
- 当定时器计数器的值TIMx_CNT达到ARR时，定时器溢出，重新从0开始向上计数，如此循环。

PWM实现原理

PWM的周期T = (ARR+1)(PSC+1)/TIMxCLK

PWM的占空比= CCRx/(ARR+1)*100%

PWM影响因素：

（1）ARR ： 决定PWM周期

（2）CCRx ： 决定PWM占空比

PWM实现原理

举例说明

假设定时器的时钟为72MHz，要求输出周期为2s，占空比为50%的PWM信号。试确定ARR、PSC、CCR的值。

解：由上述公式可设预分频系数为7199，自动重载值为19999，由50%的占空比可得CCR的值为10000。

$$\text{则周期}T = (19999 + 1) * (7199 + 1)/72000000 = 2s$$

$$\text{占空比}Duty = 10000/(19999 + 1) * 100\% = 50\%$$

PWM实现原理

注意事项

1 由定时功能衍生而来

2 每个定时器有1~4个独立通道

3 最多可同时输出30路PWM

4 通道引脚与GPIO复用

02

PWM相关的HAL库接口函数及应用

定时器的HAL库接口函数

1、PWM初始化函数（HAL_TIM_PWM_Init）

项目	函数说明
函数原型	HAL_StatusTypeDef **HAL_TIM_PWM_Init**(TIM_HandleTypeDef *htim)
功能描述	配置定时器的基础时钟和时基单元、初始化PWM模式所需的相关硬件资源
入口参数	*htim：定时器句柄的地址，取值范围是&htim1—&htim8
返回值	HAL_OK：启动成功；HAL_ERROR：启动失败
注意事项	该函数在定时器其他初始化前调用，由CubeMX软件自动生成
参考示例	HAL_TIM_PWM_Init(&htim1);　　//初始化TIM1的PWM

定时器的HAL库接口函数

2、PWM轮询方式启动函数（HAL_TIM_PWM_Start）

项目	函数说明
函数原型	HAL_StatusTypeDef **HAL_TIM_PWM_Start**(TIM_HandleTypeDef *htim, uint32_t Channel)
功能描述	轮询方式下启动PWM信号波形的输出
入口参数1	*htim：定时器句柄的地址，取值范围是&htim1—&htim8
入口参数2	Channel：捕获/比较通道，取值范围：TIM_CHANNEL_1~TIM_CHANNEL_4
返回值	HAL_OK：启动成功；HAL_ERROR：启动失败
注意事项	该函数由用户在定时器初始化完成之后调用
参考示例	HAL_TIM_PWM_Start(&htim3, TIM_CHANNEL_1); //开启TIM3的通道1输出PWM

定时器的HAL库接口函数

3、捕获/比较设置函数（__HAL_TIM_SET_COMPARE）

项目	函数说明
函数原型	__HAL_TIM_SET_COMPARE(__HANDLE__, __CHANNEL__, __COMPARE__)
功能描述	设置捕获/比较寄存器TIMx_CCRn的值CCR，控制PWM信号的占空比
宏参数1	__HANDLE__：指向定时器句柄的指针，用于指定设置哪个定时器，取值范围是&htim1—&htim8
宏参数2	__CHANNEL__：用于指定定时器的通道，取值范围：TIM_CHANNEL_1~TIM_CHANNEL_4
宏参数3	__COMPARE__：用于设置CCR的值
返回值	无
注意事项	该函数由用户调用
参考示例	__HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_3, 500);

PWM输出实验一

实验设计

本实验要求使用定时器3初始PWM信号，通过TIM3_CH3、 TIM3_CH4两个通道输出占空比可变的2路PWM波来驱动绿、蓝两种颜色的LED灯，实现LED亮度变化 （亮度由暗变亮，又从亮变暗，如此循环，即呼吸灯效果）。

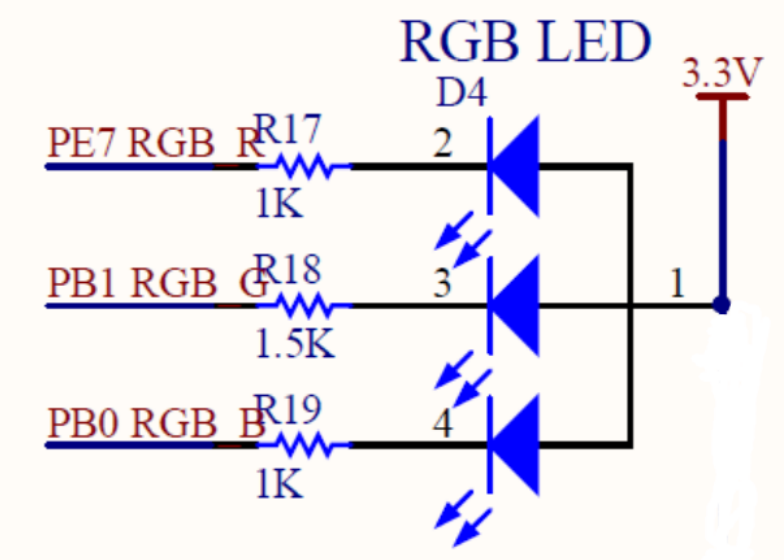
LED灯低电平点亮，高电平熄灭，占空比越大，一个周期中高电平持续时间越长，亮度越暗，反之越亮。

定时器3的4个通道对应的引脚及重映射

复用功能	TIM3_REMAP[1:0] = 00 (没有重映像)	TIM3_REMAP[1:0] = 10 (部分重映像)	TIM3_REMAP[1:0] = 11 (完全重映像)(1)
TIM3_CH1	PA6	PB4	PC6
TIM3_CH2	PA7	PB5	PC7
TIM3_CH3	PB0		PC8
TIM3_CH4	PB1		PC9

PWM输出实验一

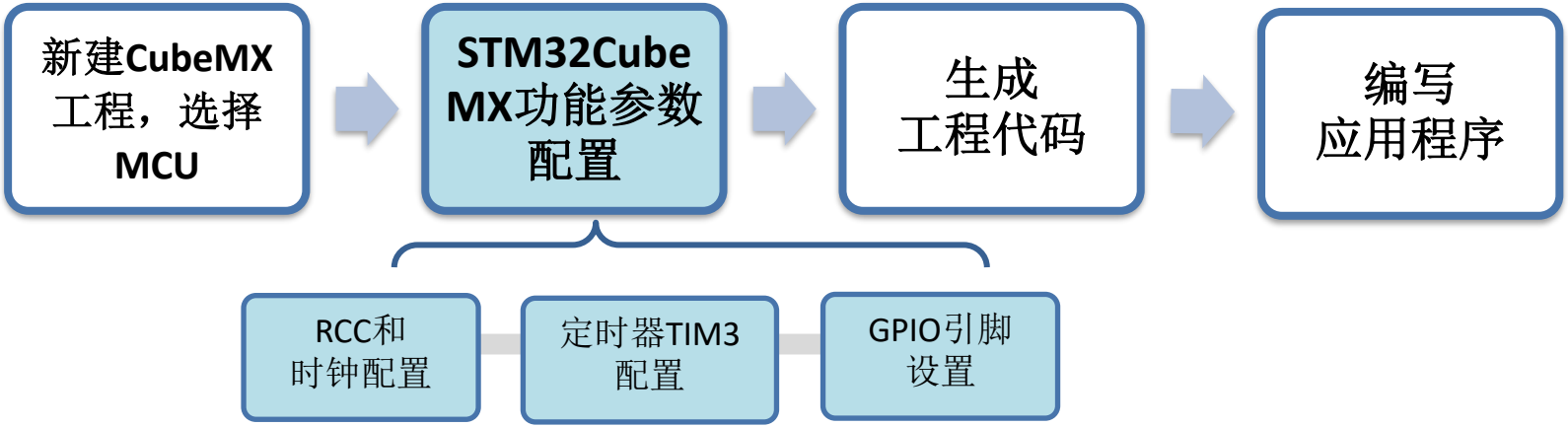
1. 硬件设计



PB0 RGB_B	35	PB0/ADC12_IN8/TIM3_CH3/TIM8_CH2N
PB1 RGB_G	36	PB1/ADC12_IN9/TIM3_CH4/TIM8_CH3N

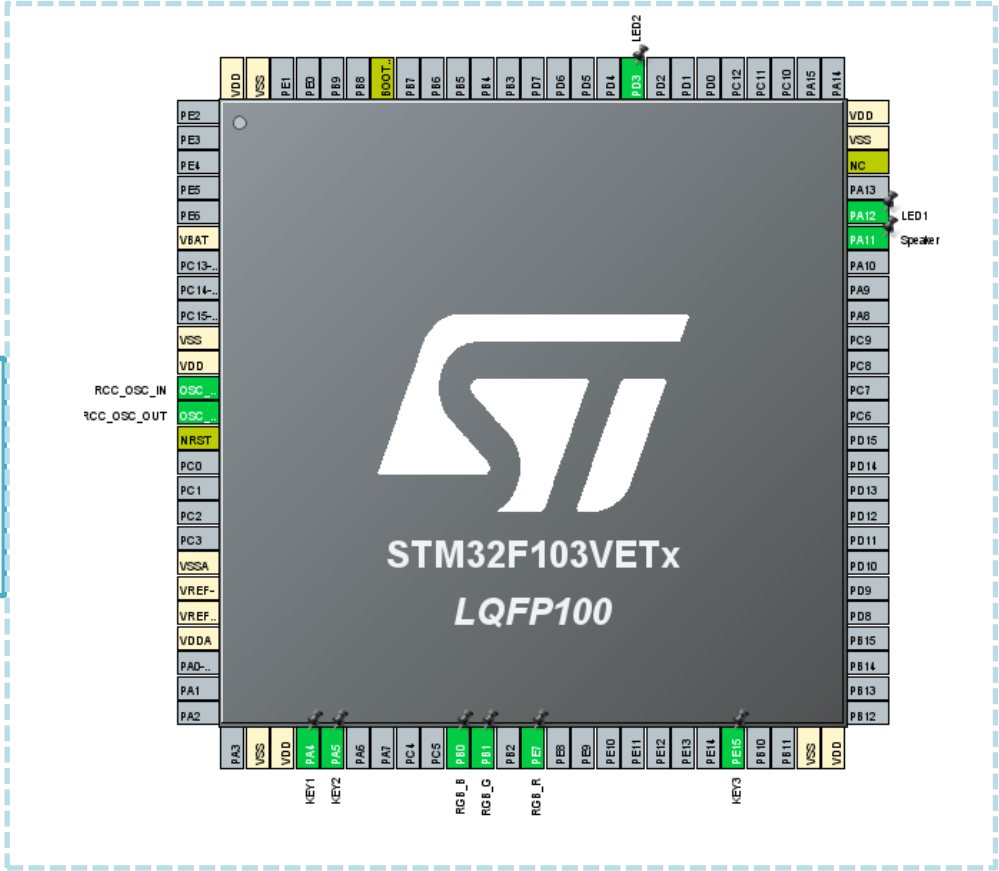
PWM输出实验一

软件设计流程



软件设计—新建STM32CubeMX工程，选择MCU

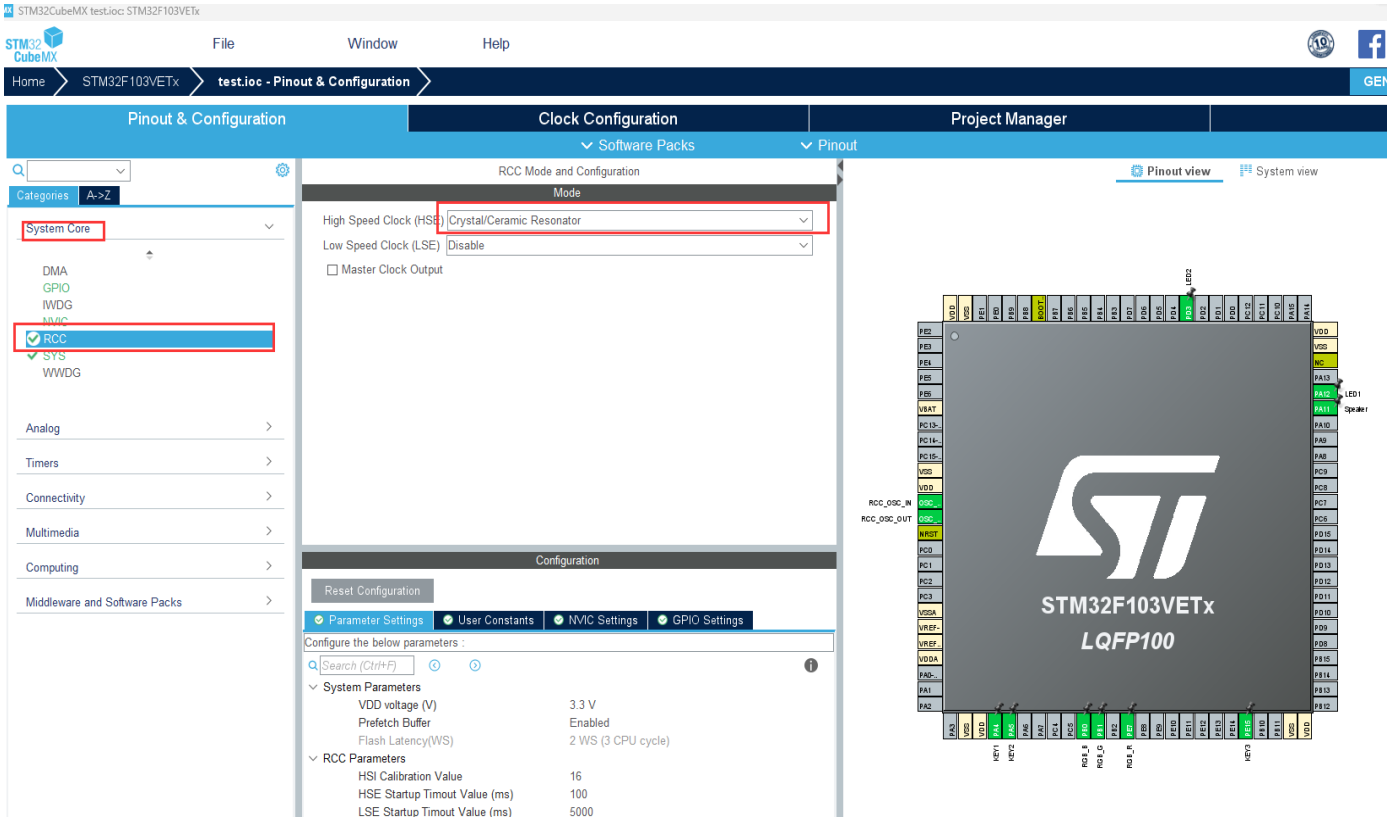
新建一个CubeMX工程，选择MCU，这里采用STM32F103VET6芯片



软件设计—STM32CubeMX功能参数配置

RCC配置

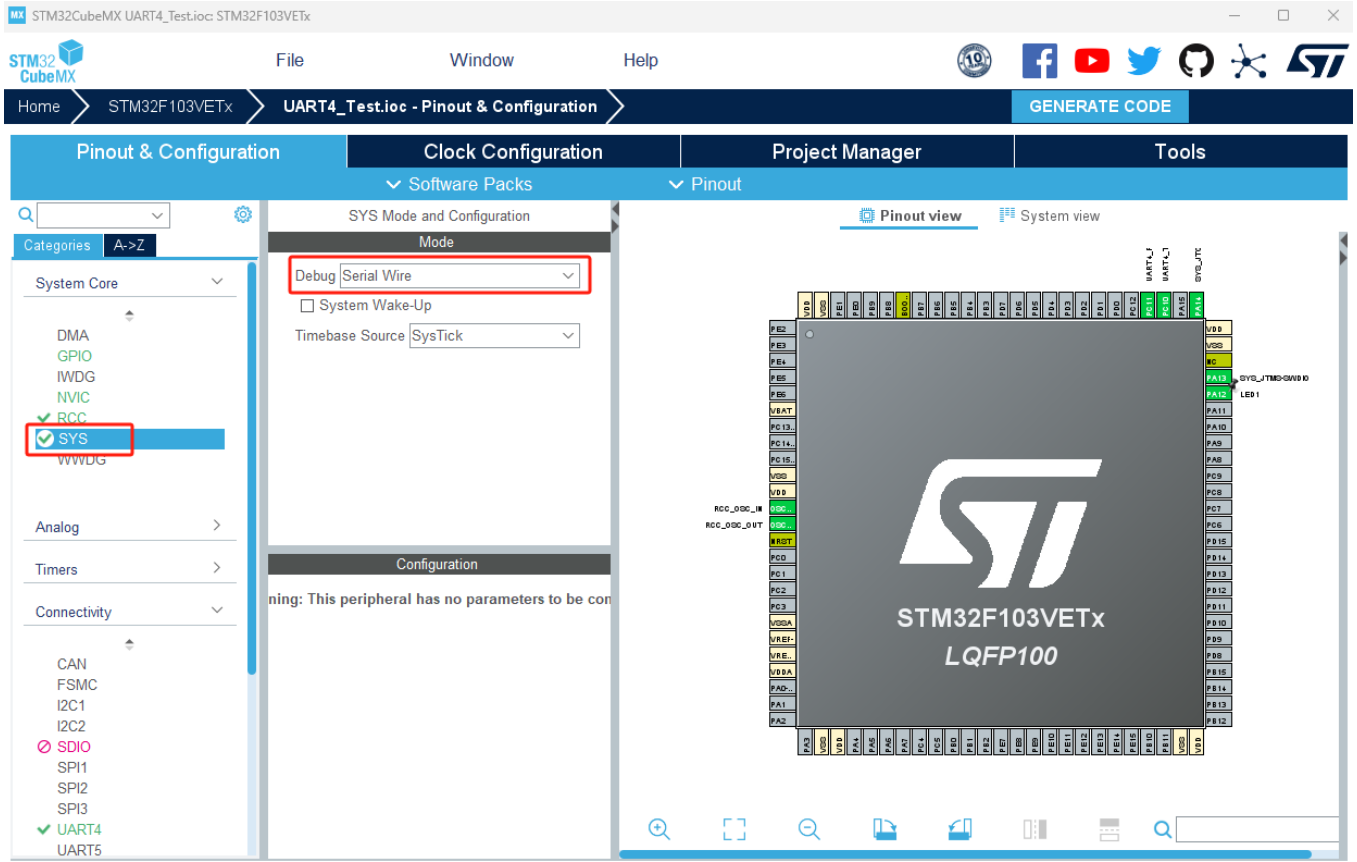
HSE选择“Crystal/Ceramic Resonator”（晶振/陶瓷谐振器），LSE选择“Disable”



软件设计—STM32CubeMX功能参数配置

SYS配置

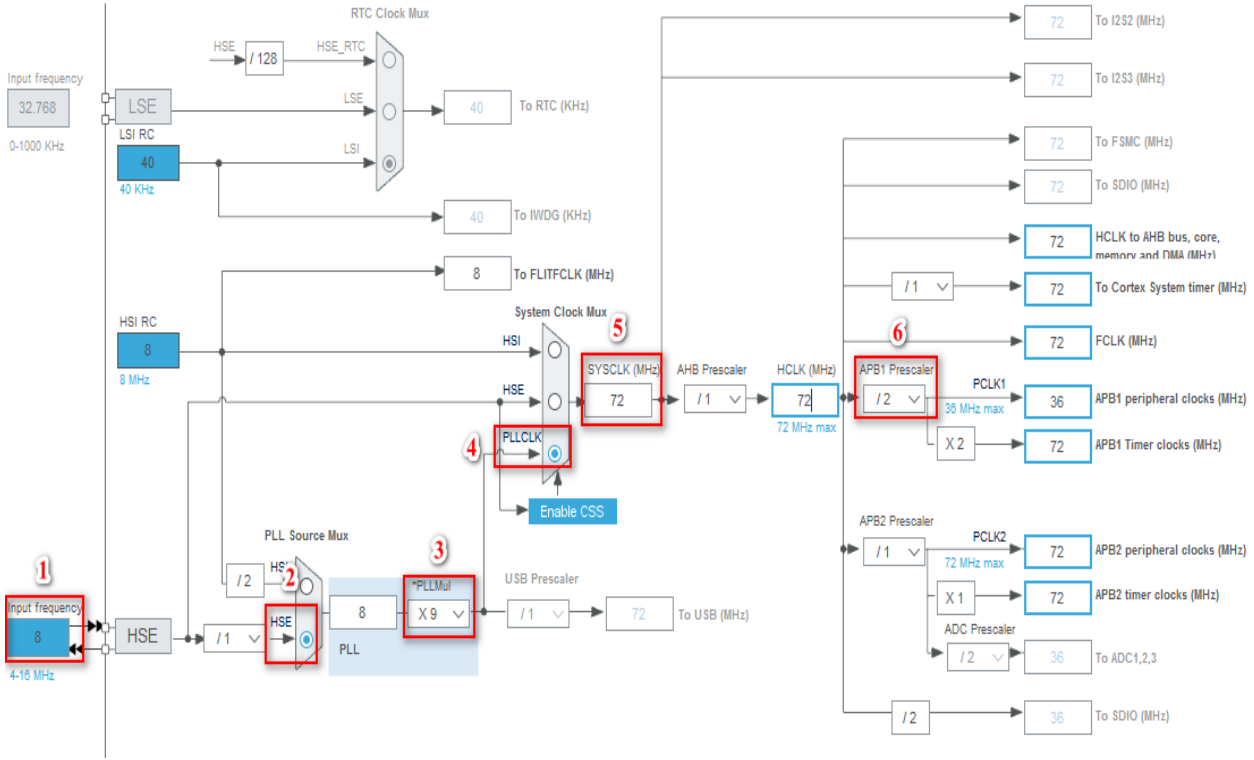
SYS模块用于配置程序的调试接口，这里Debug选择”Serial Wire”



软件设计—STM32CubeMX功能参数配置

时钟配置

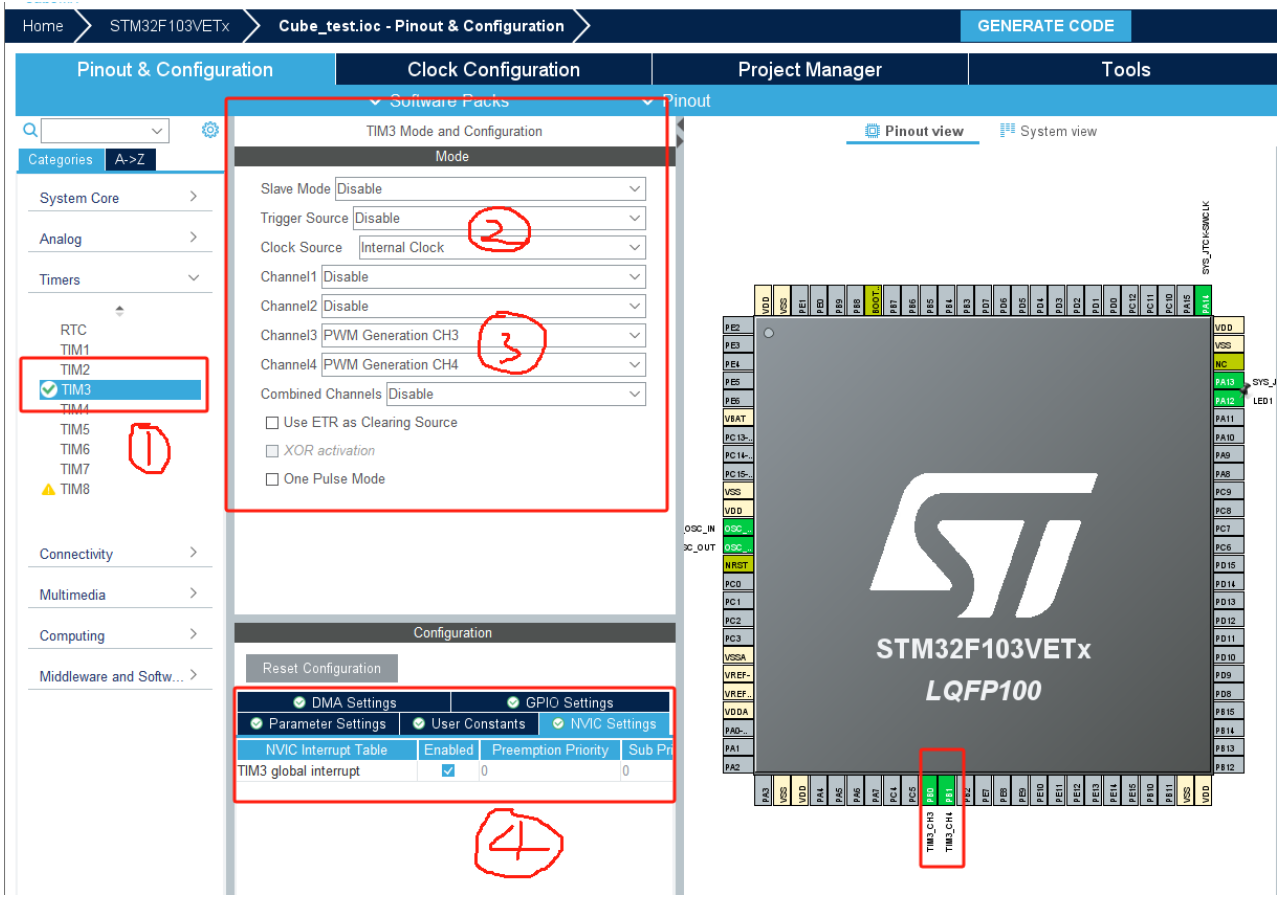
配置STM32的时钟系统，系统时钟配置为72MHz，APB2为72MHz，配置APB1为36MHz。



软件设计—STM32CubeMX功能参数配置

定时器TIM3配置

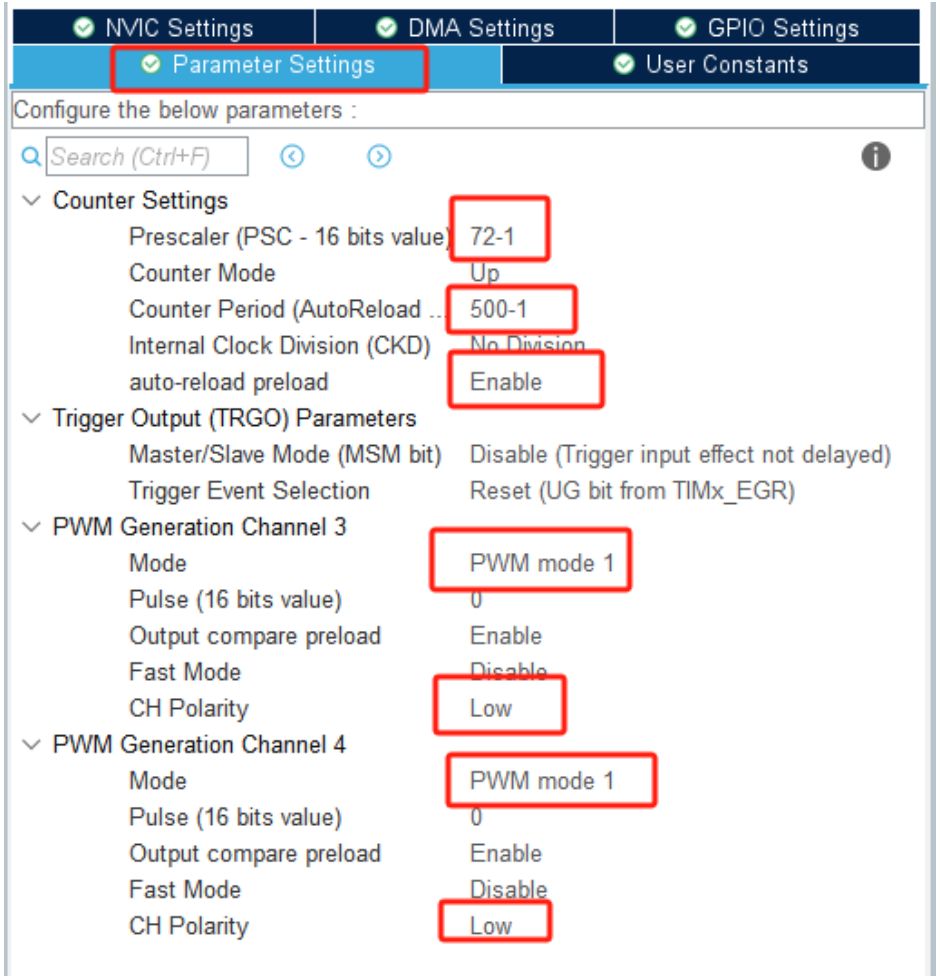
设置定时器TIM3的通道3和通道4分别为PWM Generation CH3和PWM Generation CH4，“Clock Source”设置为“Internal Clock”（内部时钟），开启TIM3 global interrupt中断。



软件设计—STM32CubeMX功能参数配置

定时器TIM3配置

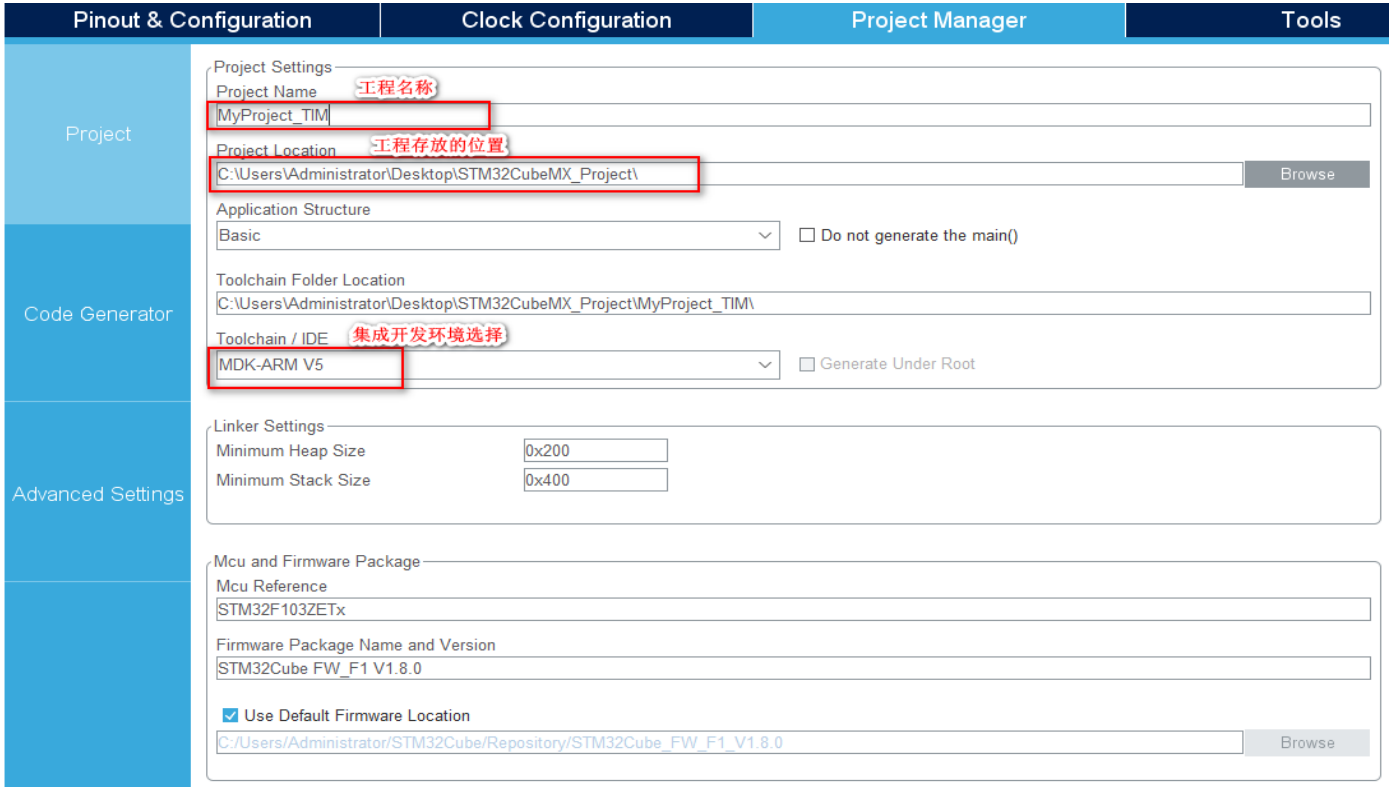
- 预分频系数“Prescaler”设置为“72-1”；“Counter Mode”设置为“Up”
- 计数值“Counter Period”设置为“500-1”
- “auto-reload preload”设置为“Enable”



软件设计—生成工程代码

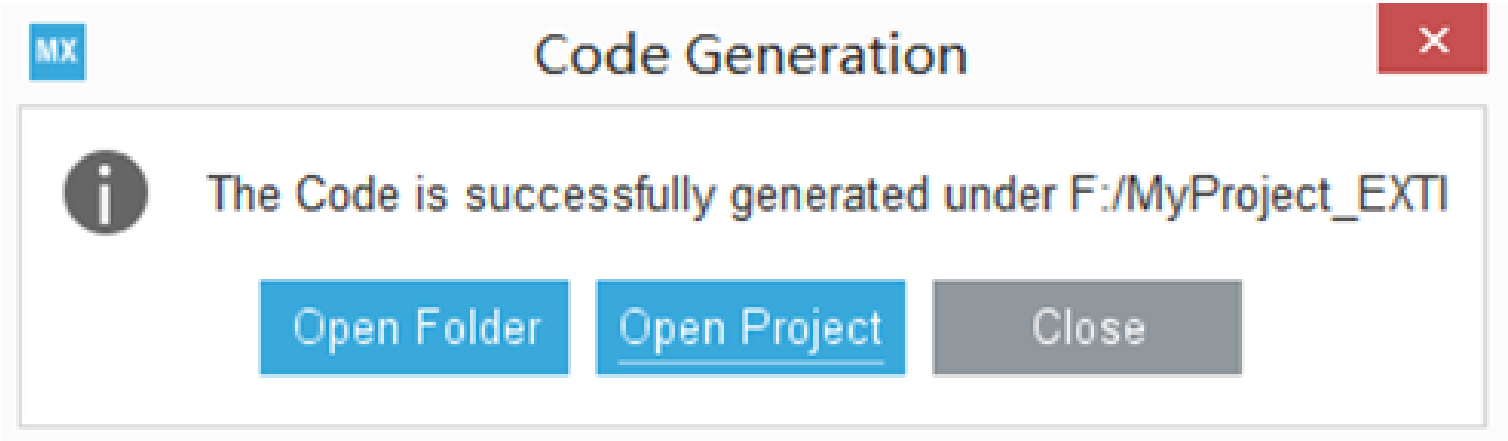
工程相关参数配置

配置工程名称、工程保存位置、选择“MDK-ARM”编译器等，生成工程代码



软件设计—生成工程代码

通过STM32CubeMX的菜单栏中的“Generate Code”生成工程代码，生成代码后，会提示是否打开该工程窗口



软件设计—编写用户程序

1、在main.c文件中的/*USER CODE BEGIN1 */和/*USER CODE END1 */之间，首先定义16位变量的PWM_Val，用于存储占空比的值。

```
/* USER CODE BEGIN 1 */

uint16_t PWM_Val=0;

/* USER CODE END 1 */
```

2、开启定时器TIM3的通道3和4输出PWM波形

```
/* USER CODE BEGIN 2 */

HAL_TIM_PWM_Start(&htim3 , TIM_CHANNEL_3);

HAL_TIM_PWM_Start(&htim3 , TIM_CHANNEL_4);

/* USER CODE END 2 */
```

软件设计—编写用户程序

在while(1)中，修改TIM3_CH3和TIM3_CH4的比较寄存器(CCR)的值（占空比）。

编写完用户程序后，点击编译按钮，然后将程序下载到开发板上进行验证，观察实验结果，是否与预期的一致。

改变PWM1和PWM2模式，观察实验现象有什么不一样。

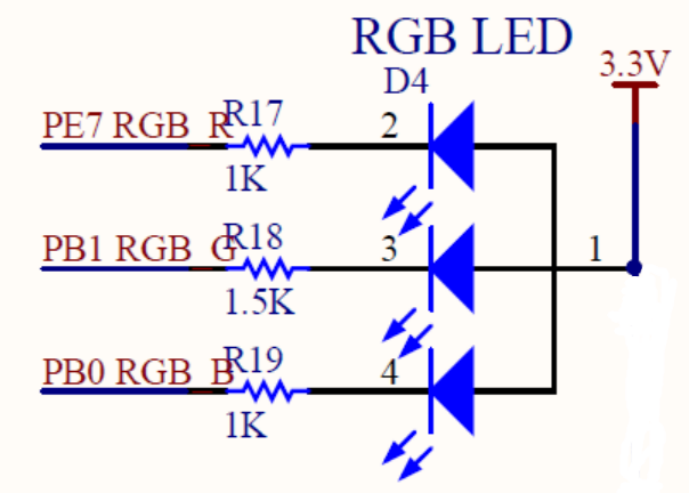
```
while (1)
 {
  /* USER CODE END WHILE */

  /* USER CODE BEGIN 3 */
  while(PWM_Val<500)
  {
                PWM_Val++;
                __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_3, PWM_Val);
                __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_4, PWM_Val);
                HAL_Delay(1);   //延时是为了更好的观察实验现象，取值1-10都行
   }
 while(PWM_Val)
  {
                PWM_Val--;
                __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_3, PWM_Val);
                __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_4, PWM_Val);
                HAL_Delay(1); //延时是为了更好的观察实验现象，取值1-10都行
   }
                HAL_Delay(200);

 }
 /* USER CODE END 3 */
```

PWM输出实验二

自主实践：通过TIM8_CH2N和TIM8_CH3N实现PWM波的输出，从而控制RGB的绿灯和蓝灯实现呼吸灯的效果。



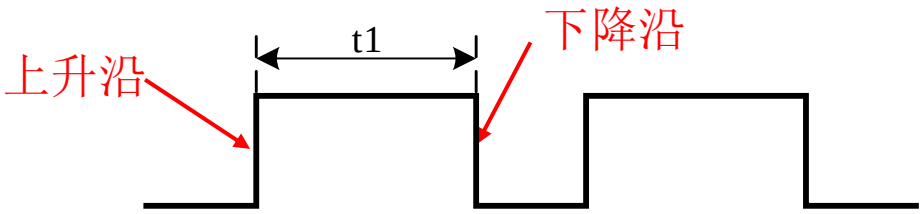
PB0 RGB_B	35	PB0/ADC12_IN8/TIM3_CH3/TIM8_CH2N
PB1 RGB_G	36	PB1/ADC12_IN9/TIM3_CH4/TIM8_CH3N

03

输入捕获简介

输入捕获简介

STM32的定时器具有捕获功能，能够捕获上升沿或者下降沿然后触发中断。

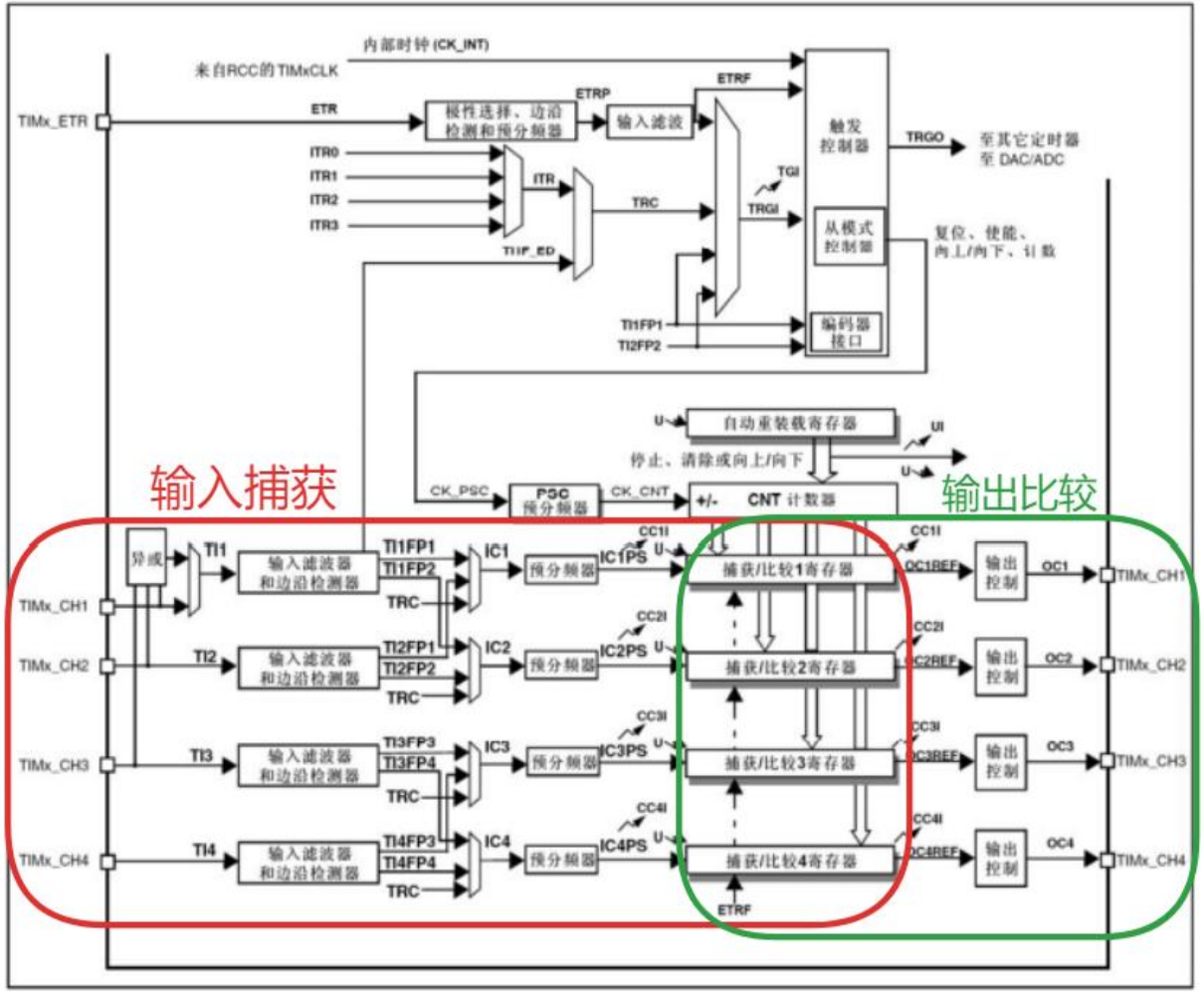


脉冲波形

实际应用：超声波测距、接收信号遥控器

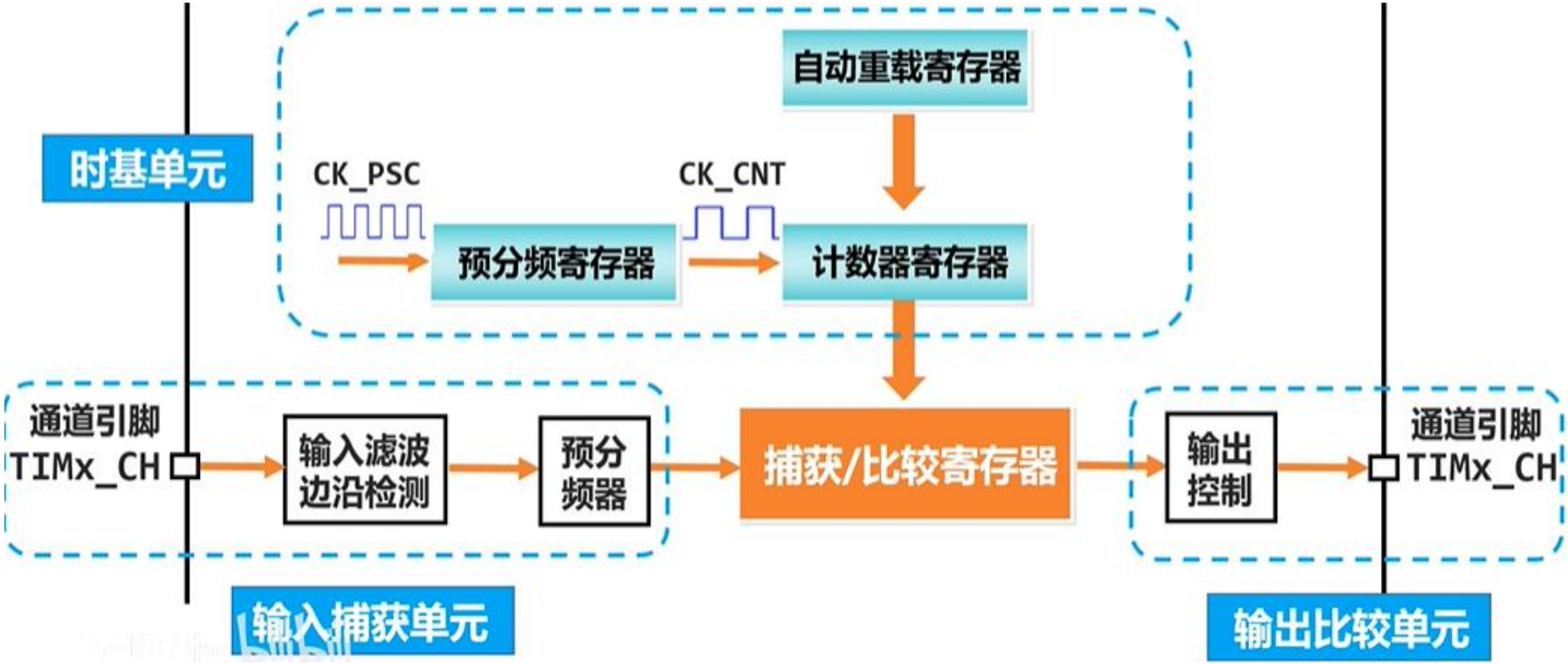
输入捕获简介

定时器框图



输入捕获简介

简化后的定时器框图



输入捕获简介

输入捕获过程总结：

得到I/O边沿 ----> 滤波 ----> 检测是上升沿还是下降沿

↓

是否要开启通道映射

↓

分频
(几次有效触发产生一起中断请求)

输入捕获简介

注意事项

1 由定时功能衍生而来

2 每个定时器有1~4个独立通道

3 捕获/比较二选一

4 通道引脚与GPIO复用

04

输入捕获的HAL库接口函数及应用

输入捕获的HAL库接口函数

1、输入捕获初始化函数 HAL_TIM_IC_Init( )

项目	函数说明
函数原型	HAL_StatusTypeDef HAL_TIM_IC_Init(TIM_HandleTypeDef *htim);
功能描述	按照参数初始化定时器的时基参数
入口参数	htim：定时器句柄的地址，取值范围是&htim1—&htim8
返回值	HAL的返回值（HAL_StatusTypeDef）； HAL_OK：启动成功；HAL_ERROR：启动失败
注意事项	**该函数由STM32CubeMX软件自动生成，用于初始化定时器的输入捕获功能。** （1）初始化定时器的输入捕获模式：该函数配置定时器硬件，使其能够工作在输入捕获模式。 （2）配置基本定时器参数：根据传入的 TIM_HandleTypeDef 结构体参数，设置定时器的时基（预分频器、自动重装载值等）和输入捕获通道（极性、滤波、分频等）。 （3）底层硬件初始化：通过 MSP 回调函数（如 HAL_TIM_IC_MspInit()）初始化与定时器相关的底层硬件（时钟、GPIO、中断等）。

输入捕获的HAL库接口函数

2、输入捕获中断方式启动函数 HAL_TIM_IC_Start_IT（）

项目	函数说明
函数原型	HAL_StatusTypeDef **HAL_TIM_IC_Start_IT** (TIM_HandleTypeDef *htim, uint32_t Channel);
功能描述	在中断方式下启动定时器对输入信号的捕获
入口参数1	***htim**：定时器句柄的地址，取值范围是**&htim1—&htim8**
入口参数2	**Channel**：定时器通道号，取值范围是**TIM_CHANNEL_1—TIM_CHANNEL_4**
返回值	HAL的返回值（HAL_StatusTypeDef）； HAL_OK：启动成功；HAL_ERROR：启动失败
注意事项	（1）该函数在定时器初始化完成之后调用 （2）函数需要由用户调用，用于使能定时器的捕获中断，并启动定时器运行

输入捕获的HAL库接口函数

3、输入捕获中断方式停止函数
HAL_TIM_IC_Stop_IT（）

项目	函数说明
函数原型	HAL_StatusTypeDef **HAL_TIM_IC_Stop_IT** (TIM_HandleTypeDef *htim, uint32_t Channel);
功能描述	停止中断方式下定时器对输入信号的捕获
入口参数1	***htim**：定时器句柄的地址，取值范围是**&htim1—&htim8**
入口参数2	**Channel**：定时器通道号，取值范围是**TIM_CHANNEL_1—TIM_CHANNEL_4**
返回值	HAL的返回值（HAL_StatusTypeDef）； HAL_OK：启动成功；HAL_ERROR：启动失败
注意事项	该函数需要由用户调用，用于禁止捕获中断，关闭输入捕获通道，停止定时器运行

输入捕获的HAL库接口函数

4、捕获中断回调函数
HAL_TIM_IC_CaptureCallback（）

项目	函数说明
函数原型	void HAL_TIM_IC_CaptureCallback (TIM_HandleTypeDef *htim)
功能描述	该函数用来处理发生捕获中断后的具体任务，任一定时器发生捕获中断后，都会调用该函数。用户在该函数内编写实际的任务处理程序。
入口参数	*htim：定时器句柄的地址，取值范围是&htim1—&htim8
返回值	无
注意事项	该函数由定时器中断通用处理函数HAL_TIM_IRQHandler( )调用，在函数内部首先需要判断是哪一个定时器产生的本次捕获中断，中断的具体任务由用户根据要求编写。

输入捕获的HAL库接口函数

5、捕获边沿设置函数
__HAL_TIM_SET_CAPTUREPOLARITY（）

项目	函数说明
函数原型	__HAL_TIM_SET_CAPTUREPOLARITY(__HANDLE__, __CHANNEL__, __POLARITY__)
功能描述	设置输入信号的捕获边沿
入口参数1	__HANDLE__：指向定时器句柄的地址（即定时器句柄的地址）
入口参数2	__CHANNEL__：定时器通道号，取值范围是TIM_CHANNEL_1—TIM_CHANNEL_4
入口参数3	__POLARITY__：捕获边沿 TIM_INPUTCHANNELPOLARITY_RISING：上升沿捕获 TIM_INPUTCHANNELPOLARITY_FALLING：下降沿捕获 TIM_INPUTCHANNELPOLARITY_BOTHEDGE ：双边沿捕获
返回值	无
注意事项	该函数由用户调用，用于设置输入信号的有效捕获边沿

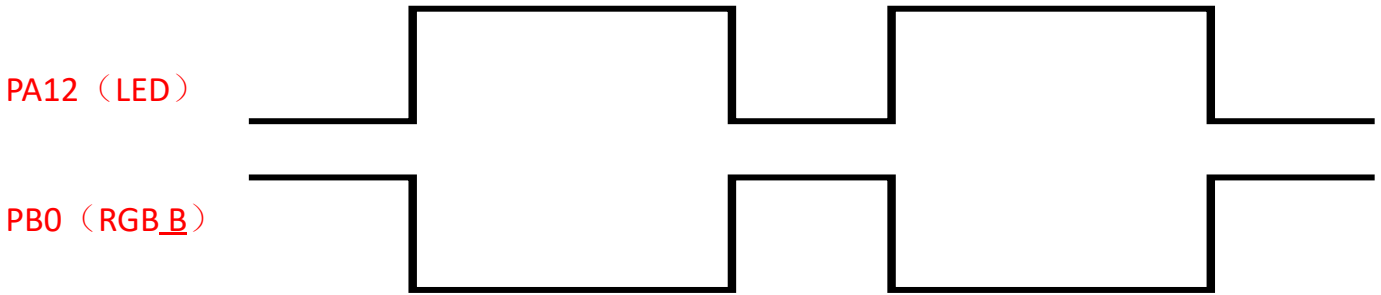
输入捕获的HAL库接口函数

6、捕获值读取函数
HAL_TIM_ReadCapturedValue（）

项目	函数说明
函数原型	**uint32_t HAL_TIM_ReadCapturedValue(TIM_HandleTypeDef *htim, uint32_t Channel);**
功能描述	用于读取捕获/比较寄存器TIMx_CCR的值，即捕获值
入口参数1	***htim**：定时器句柄的地址，取值范围是**&htim1—&htim8**
入口参数2	**Channel**：定时器通道号，取值范围是**TIM_CHANNEL_1—TIM_CHANNEL_4**
返回值	捕获值
注意事项	该函数需要由用户调用，用于读取发生捕获时的捕获值

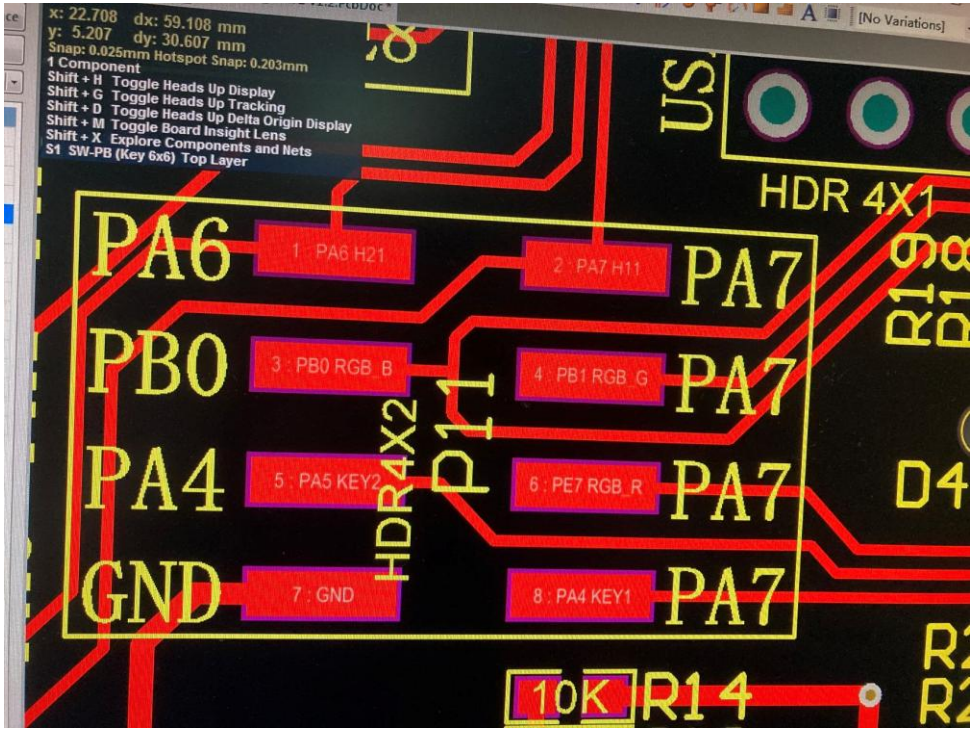
输入捕获实验

本实训要求PA12随意延时驱动LED1灯闪烁，并且将PA12与PA7端口通过杜邦线连接到一起。PA7是通用定时器TIM3的通道2，在TIM3_CH2捕获到边沿信号触发中断后，在中断服务函数中实现：让PB0口的RGB蓝灯状态取反，输出波形刚好与PA12口的波形互补（如图所示），并且在每次进入中断后改变触发捕获的极性。看到的实验现象为：两个LED灯交替闪烁。

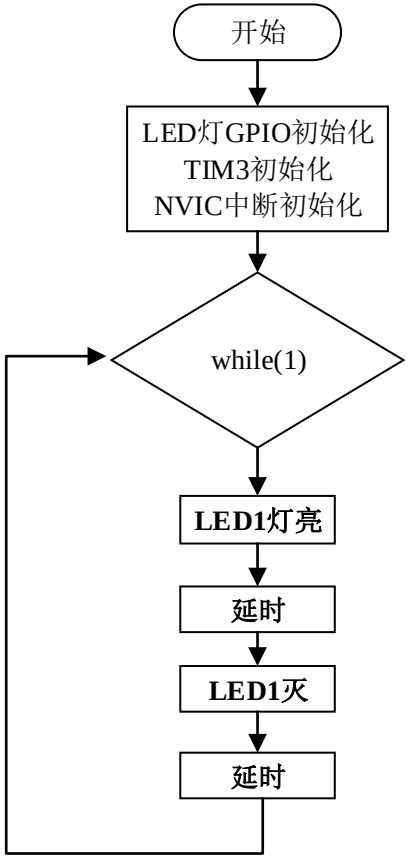


输入捕获实验

1. 硬件设计



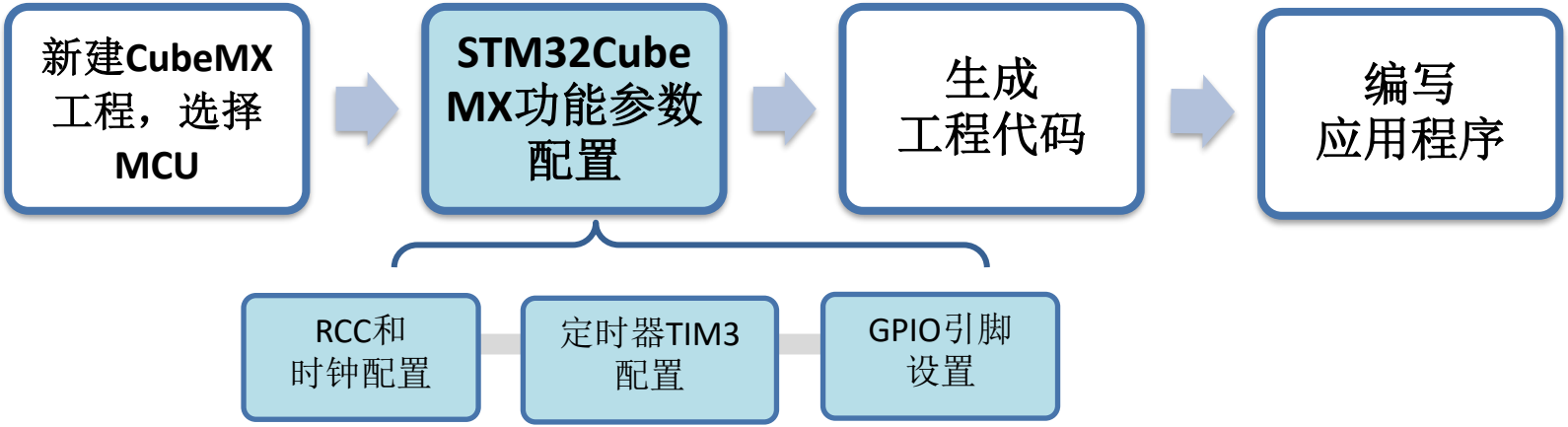
主程序实现PA12端口控制LED1灯亮灭



主程序流程图

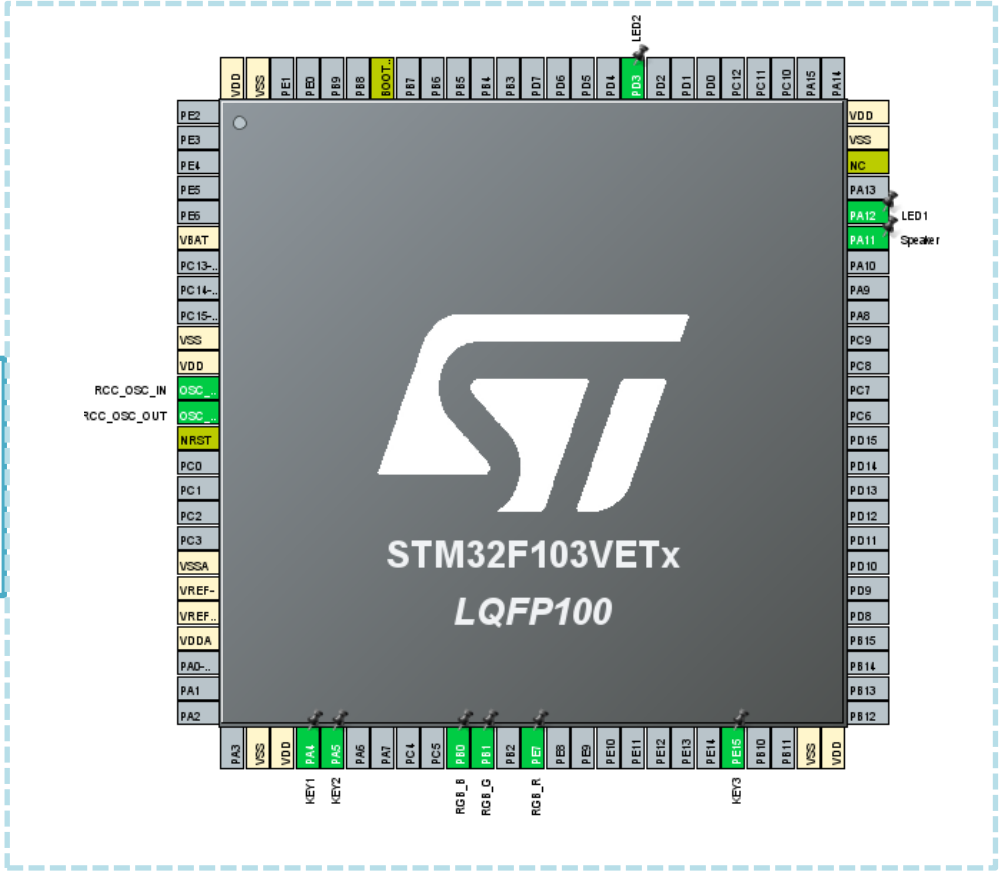
输入捕获实验

软件设计流程



软件设计—新建STM32CubeMX工程，选择MCU

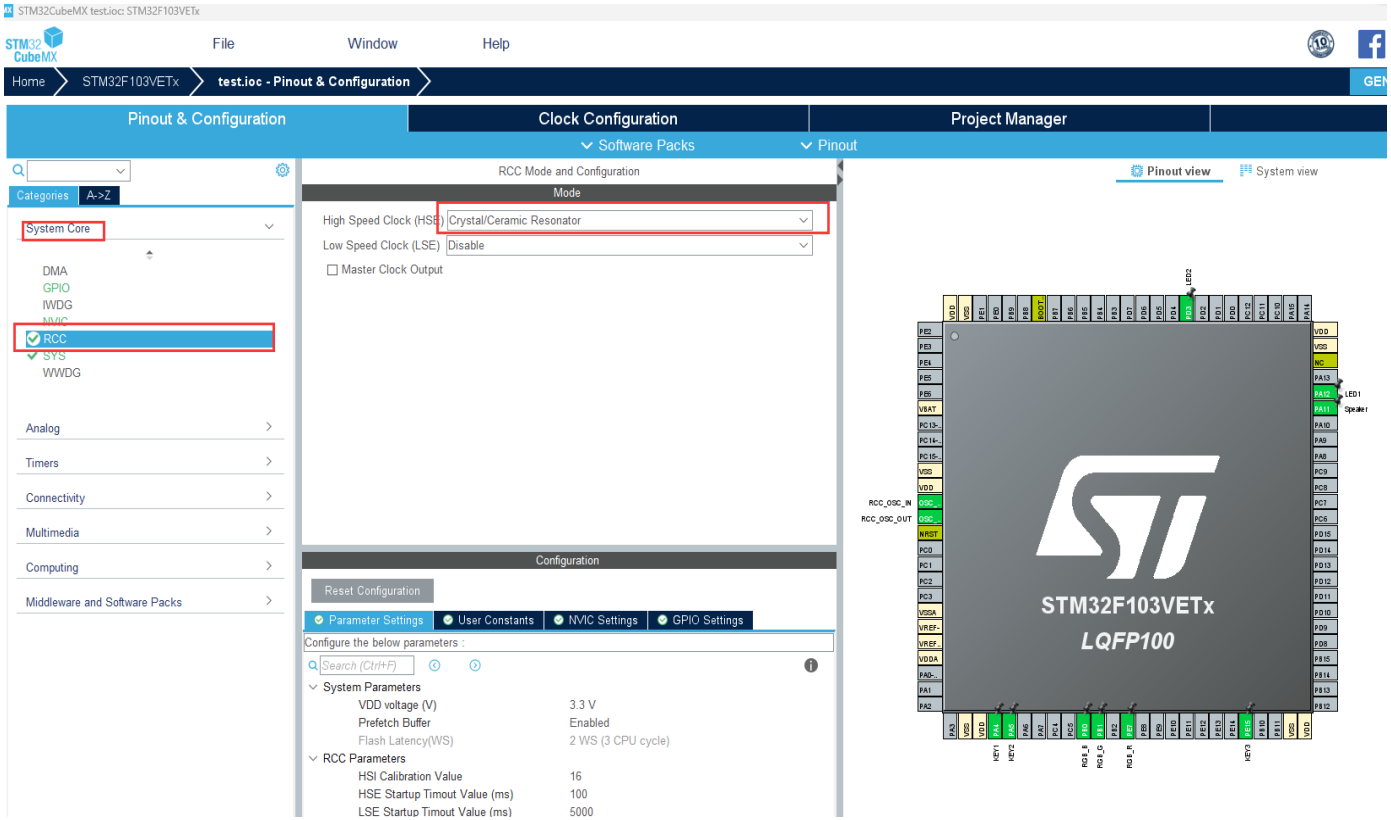
新建一个CubeMX工程，选择MCU，这里采用STM32F103VET6芯片



软件设计—STM32CubeMX功能参数配置

RCC配置

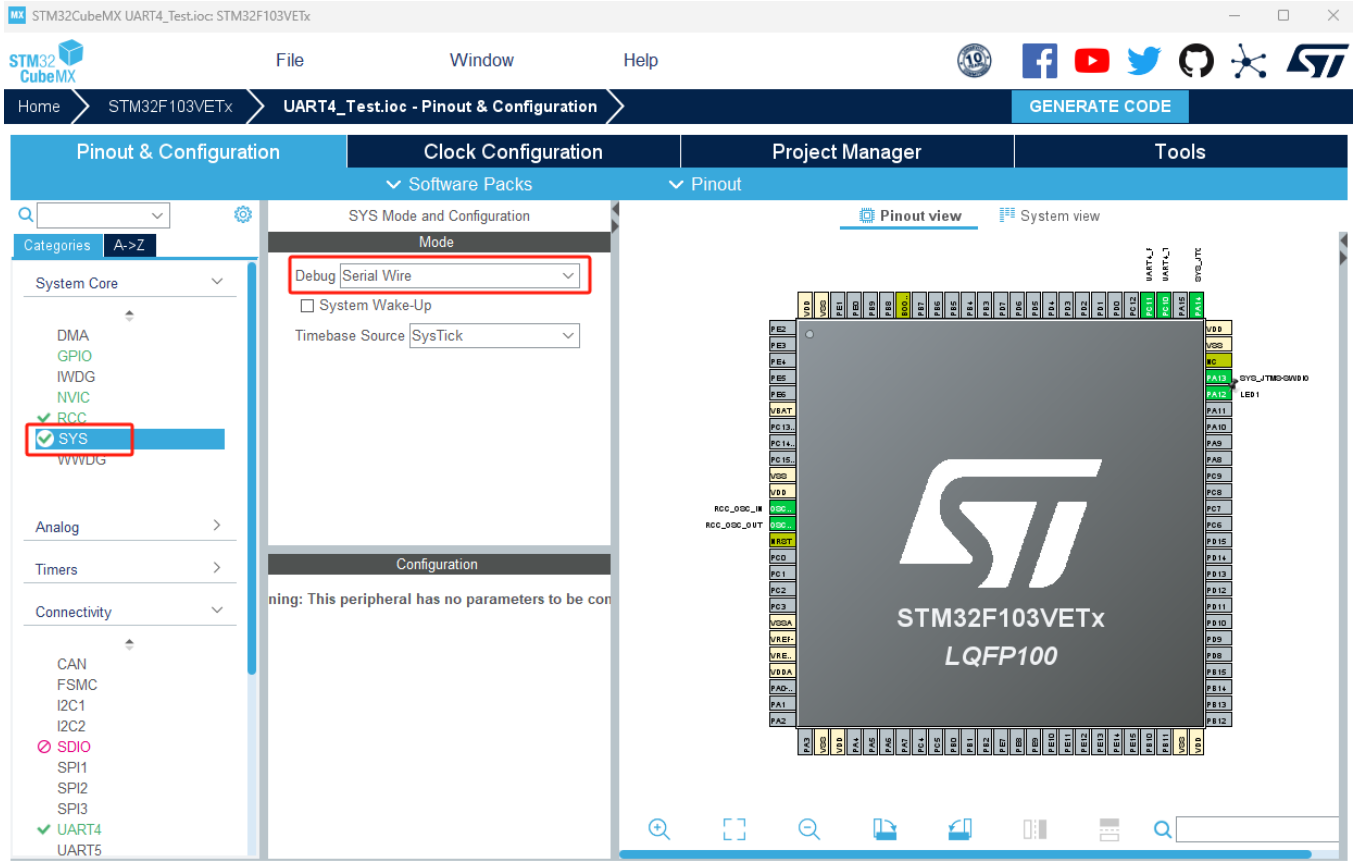
HSE选择“Crystal/Ceramic Resonator”（晶振/陶瓷谐振器），LSE选择“Disable”



软件设计—STM32CubeMX功能参数配置

SYS配置

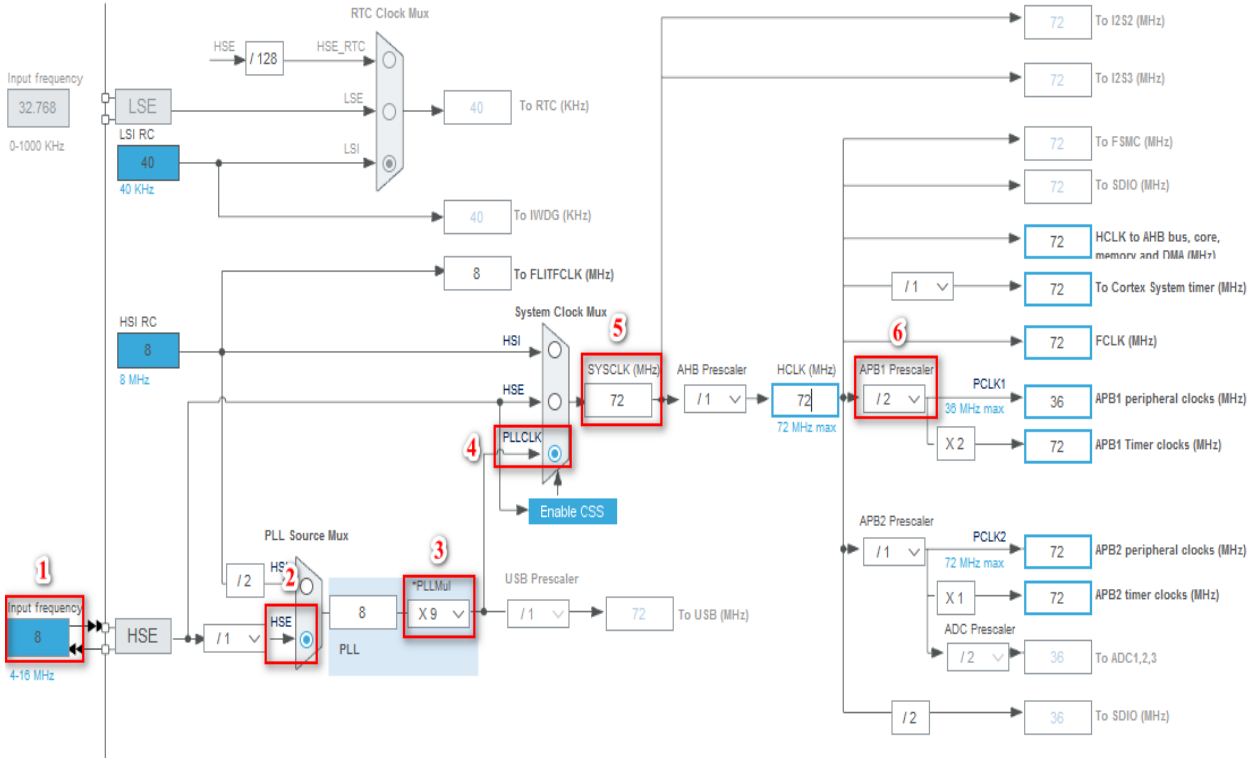
SYS模块用于配置程序的调试接口，这里Debug选择”Serial Wire”



软件设计—STM32CubeMX功能参数配置

时钟配置

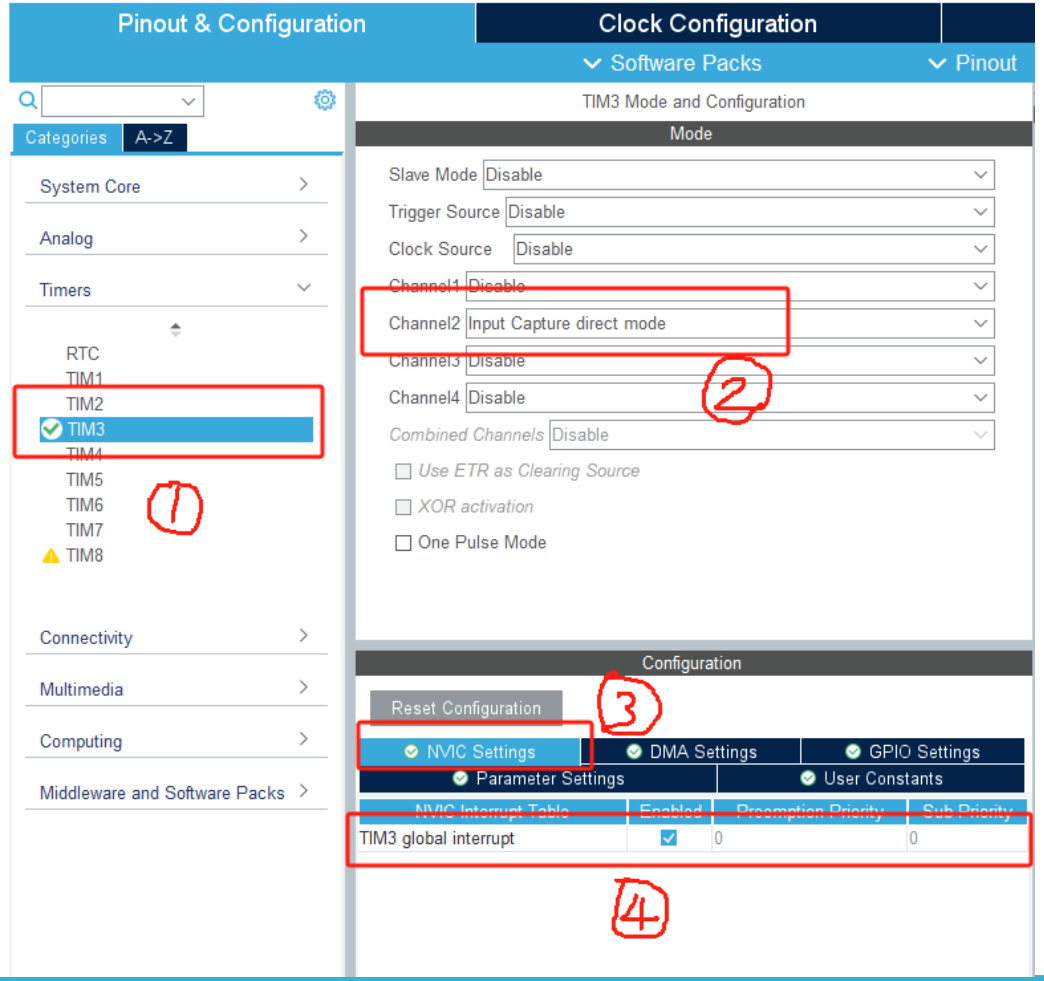
配置STM32的时钟系统，系统时钟配置为72MHz，APB2为72MHz，配置APB1为36MHz。



软件设计—STM32CubeMX功能参数配置

定时器TIM3配置

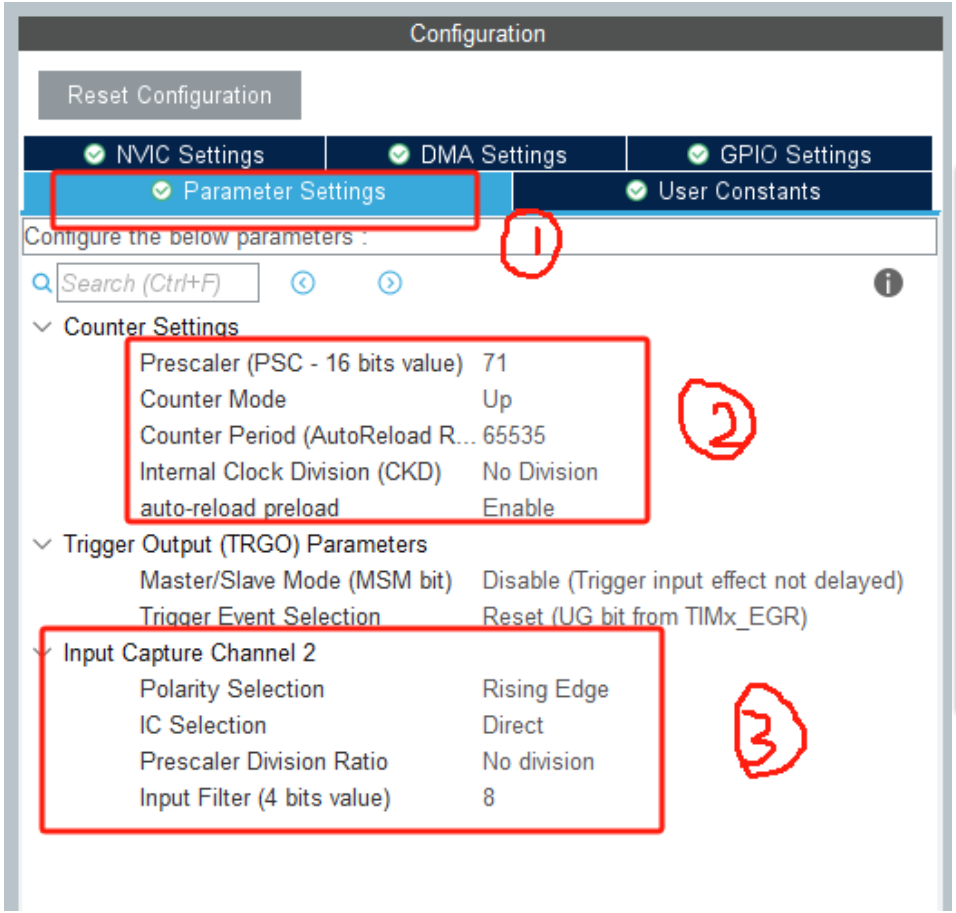
- 设置定时器TIM3的通道2为“Input Capture direct mode”
- 开启TIM3 global interrupt中断。



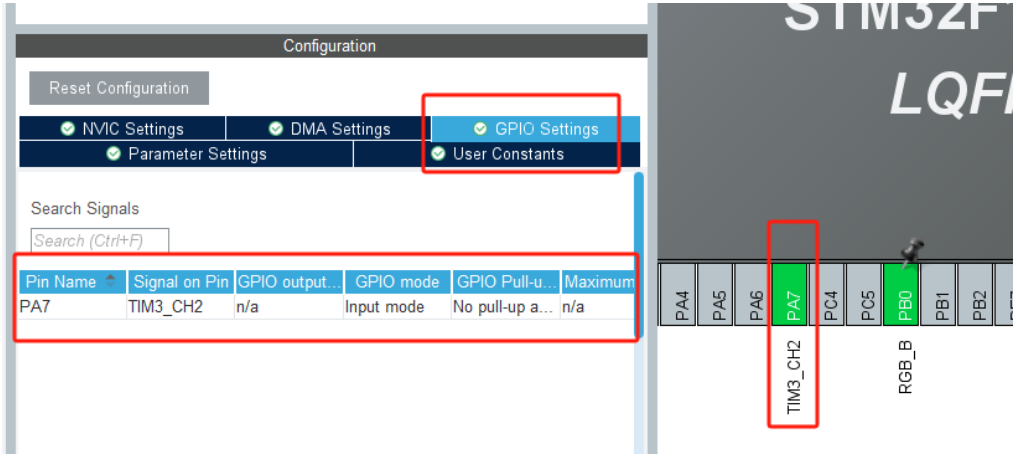
软件设计—STM32CubeMX功能参数配置

定时器TIM3配置

- 预分频系数“Prescaler”设置为“72-1”；“Counter Mode”设置为“Up”
- 计数值“Counter Period”设置为“65535”
- “auto-reload preload”设置为“Enable”

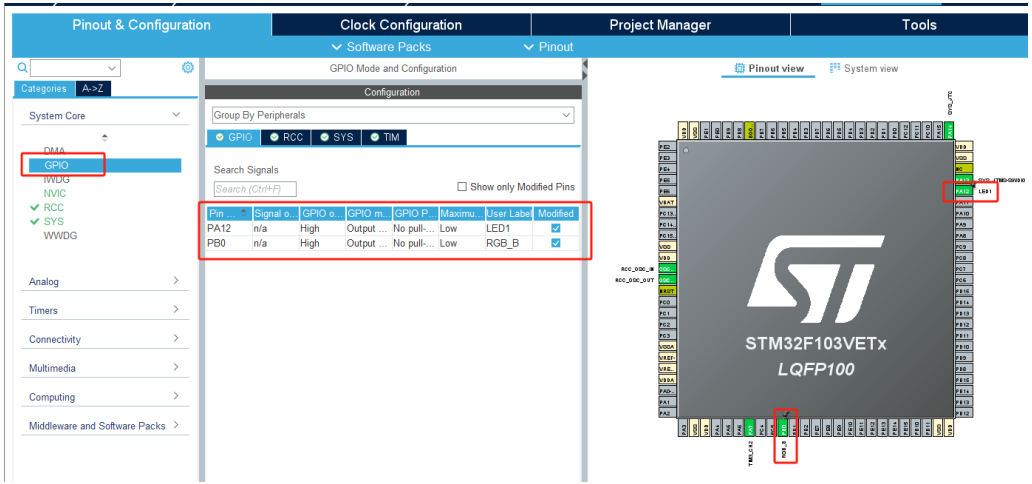


软件设计—STM32CubeMX功能参数配置



GPIO引脚配置

- PA7引脚配置为输入捕获引脚
- 配置GPIO引脚输出，用于控制LED灯的亮灭



软件设计—生成工程代码

工程相关参数配置

配置工程名称、工程保存位置、选择“MDK-ARM”编译器等，生成工程代码

Pinout & Configuration | Clock Configuration | Project Manager | Tools

Project

Code Generator

Advanced Settings

Project Settings

Project Name 工程名称

MyProject_TIM

Project Location 工程存放的位置

C:\Users\Administrator\Desktop\STM32CubeMX_Project\ Browse

Application Structure

Basic ☐ Do not generate the main()

Toolchain Folder Location

C:\Users\Administrator\Desktop\STM32CubeMX_Project\MyProject_TIM\

Toolchain / IDE 集成开发环境选择

MDK-ARM V5 ☐ Generate Under Root

Linker Settings

Minimum Heap Size 0x200

Minimum Stack Size 0x400

Mcu and Firmware Package

Mcu Reference

STM32F103ZETx

Firmware Package Name and Version

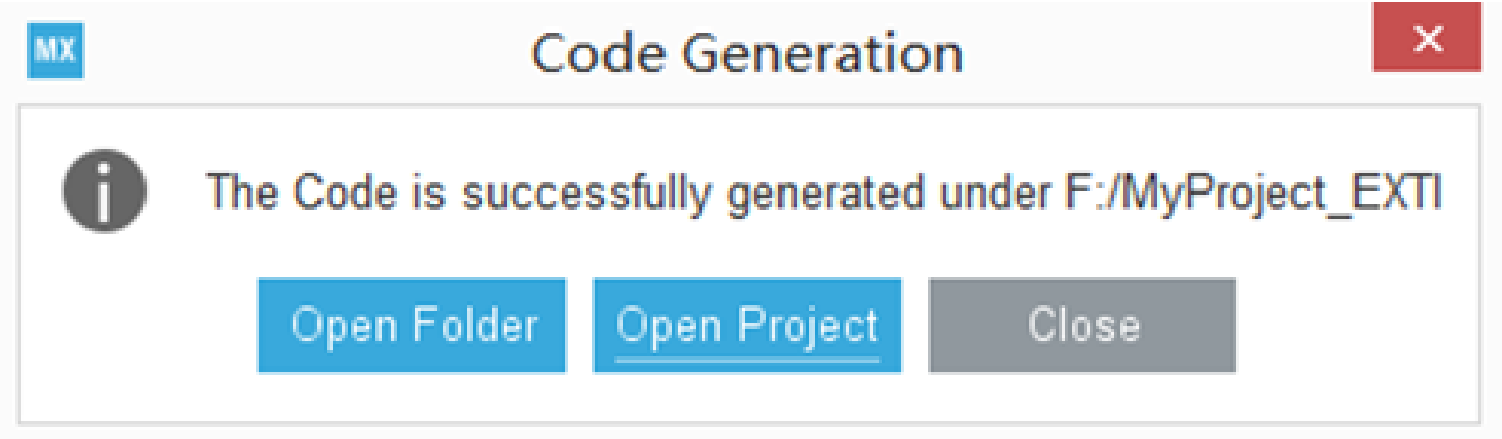
STM32Cube FW_F1 V1.8.0

☑ Use Default Firmware Location

C:/Users/Administrator/STM32Cube/Repository/STM32Cube_FW_F1_V1.8.0 Browse

软件设计—生成工程代码

通过STM32CubeMX的菜单栏中的“Generate Code”生成工程代码，生成代码后，会提示是否打开该工程窗口



软件设计—编写用户程序

1、在main.c文件中的/*USER CODE BEGIN0 */和/*USER CODE END0 */之间，首先定义一个flag标志标量，用于判断RGB蓝灯亮或者灭,同时改变捕获的边沿。

```
/* USER CODE BEGIN 0 */

uint8_t flag=0;

/* USER CODE END 0 */
```

2、在main.c文件中的/*USER CODE BEGIN2 */和/*USER CODE END2 */之间加入如下代码，开启定时器TIM3的输入捕获功能

```
/* USER CODE BEGIN 2 */

HAL_TIM_IC_Start_IT(&htim3, TIM_CHANNEL_2);

/* USER CODE END 2 */
```

软件设计—编写用户程序

3、在while(1)中，让PA12控制的LED1灯，以1S间隔循环闪烁。

```
while (1)
 {
  /* USER CODE END WHILE */

  /* USER CODE BEGIN 3 */
     HAL_GPIO_WritePin(GPIOA,GPIO_PIN_12, GPIO_PIN_RESET);
    HAL_Delay(1000);
    HAL_GPIO_WritePin(GPIOA,GPIO_PIN_12, GPIO_PIN_SET);
    HAL_Delay(1000);
 }
 /* USER CODE END 3 */
```

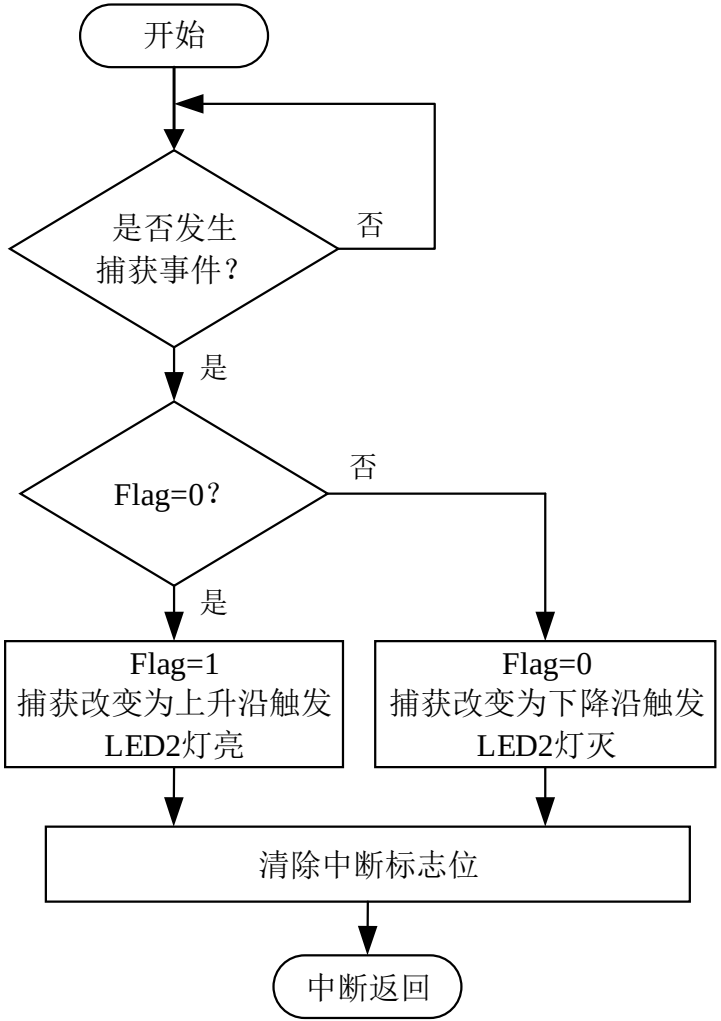
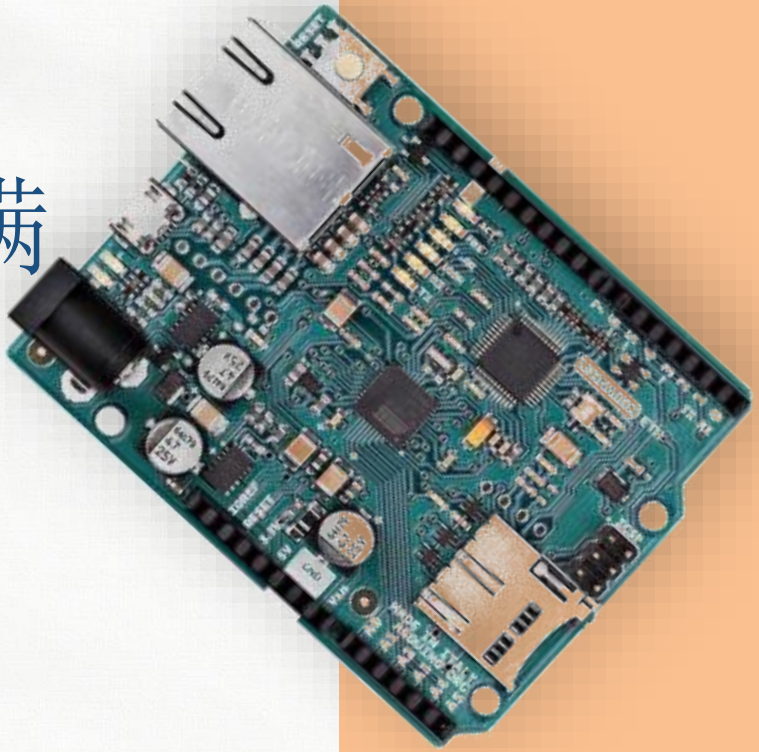
软件设计—编写用户程序

4、在main.c文件中的/*USER CODE BEGIN4 */和/*USER CODE END4 */之间加入如下代码，实现输入捕获中断的业务逻辑。

```
void HAL_TIM_IC_CaptureCallback(TIM_HandleTypeDef *htim)
{
    if(htim->Instance == htim3.Instance)
    {
        if(flag==0)
        {
          flag=1;
           __HAL_TIM_SET_CAPTUREPOLARITY(&htim3,TIM_CHANNEL_2,TIM_ICPOLARITY_FALLING);
           //设置为下降沿捕获
          HAL_GPIO_WritePin(GPIOB,GPIO_PIN_0, GPIO_PIN_RESET); //RGB蓝灯亮
        }
        else
        {
          flag=0;
           __HAL_TIM_SET_CAPTUREPOLARITY(&htim3,TIM_CHANNEL_2,TIM_INPUTCHANNELPOLARITY_RISING);
           //设置为上升沿捕获
            HAL_GPIO_WritePin(GPIOB,GPIO_PIN_0, GPIO_PIN_SET);  //RGB蓝灯灭
        }
    }
}
```

输入捕获实验总结

中断服务函数实现当捕获到PA12输出的边沿时，通过变量Flag标志变量判断RGB蓝灯亮或者灭,同时改变捕获的边沿。程序流程图如右图所示。



本章结束，收获满满